ENSEA
Ecole Nationale Supérieure de l'Electronique et de ses Applications

MODELISATION ET IMPLEMENTATION D'UNE FIFO ÉLASTIQUE SYNCHRONE

Objectifs :

Méthodologie & Aspects avancés de VHDL
Modélisation par Paramètres génériques,
Mémoires, Compteurs, Séquenceurs,
Paquetages, Assertions, Attributs,
Procédures et Fonctions,
Simulation par Configuration
Synthèse

La table des matières est disponible à la fin du document (page 13)

Cahier des charges

L'objectif est de concevoir et modéliser une FIFO élastique synchrone. Les composants de base qui vont la constituer et le fonctionnement requis par ce cahier des charges permettront de voir un certain nombre d'aspects avancés de VHDL. La particularité de cette mémoire FIFO c'est que l'écriture des données se fait d'une manière **asynchrone** (sous le contrôle d'un protocole (requête - acquittement) mais la lecture est prioritaire et se fait d'une manière **synchrone** synchronisé par une horloge nommée Hl.

- La taille de la FIFO est paramétrable en nombre de mots et nombre de bits par mot.
- Le séquencement interne est synchrone et réalisé à partir d'une horloge externe CLK de 10 Mhz.
- La fréquence des lectures Hl est de 50 kHz générée à partir de l'horloge de base de CLK.
- Les données sont positionnées en sortie sur le front montant de Hl et entre deux lectures le bus de données en sortie doit être en haute impédance.

- La priorité de la lecture fait que l'acquittement de l'écriture Ack doit être retardée si une lecture est en cours.
- Afin d'éviter que la FIFO se vide ou déborde, les 2 signaux ***fast*** et ***slow*** sont générés. Dans le cas d'une FIFO 16x8 (16 mots de 8 bits), le signal ***fast*** est positionné à 1 si la FIFO contient **moins** de **4 données**, de même, le signal ***slow*** est positionné à 1 si la fifo contient **12 données ou plus**. Ces valeurs sont valables uniquement pour une fifo 16x8. Dans le cas général, on prendra les conditions suivantes :

  Fast vaut 1 si le nombre de mots dans la fifo est $< 2^{M-2}$

  Slow vaut 1 si le nombre de mots dans la fifo est $\geq 2^{M}-2^{M-2}$.

  2^{M} étant le nombre de mots max.

- Le séquencement interne s'effectue à l'aide d'un séquenceur possédant 2 architectures (sorties Moore et sorties Mealy). Le choix de l'architecture du séquenceur lors de la simulation de la FIFO se fait à l'aide d'un fichier de configuration.
- Les données sont converties en code complément à 2 avant d'être sauvegardées (composant fourni).

- Une vérification permanente des temps pré positionnement et temps de maintien dans les éléments de mémorisation comme la RAM ou le registre (procédures fournies).

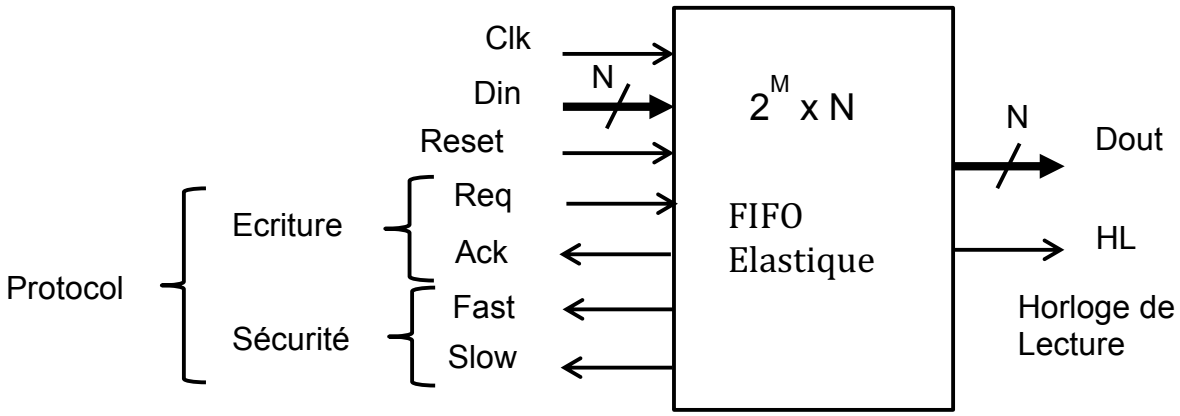


Figure 1

Un scénario parmi d'autres

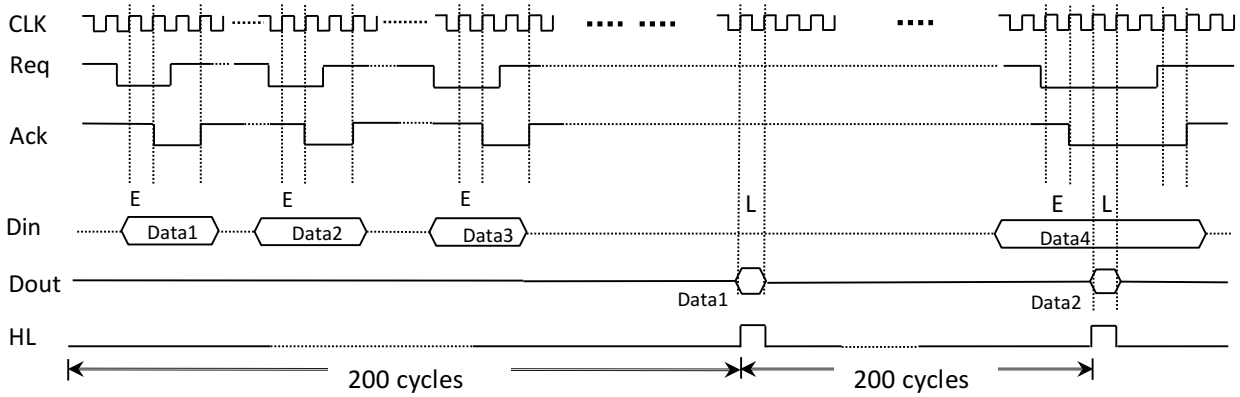


Figure 2

Un autre scénario parmi d'autres

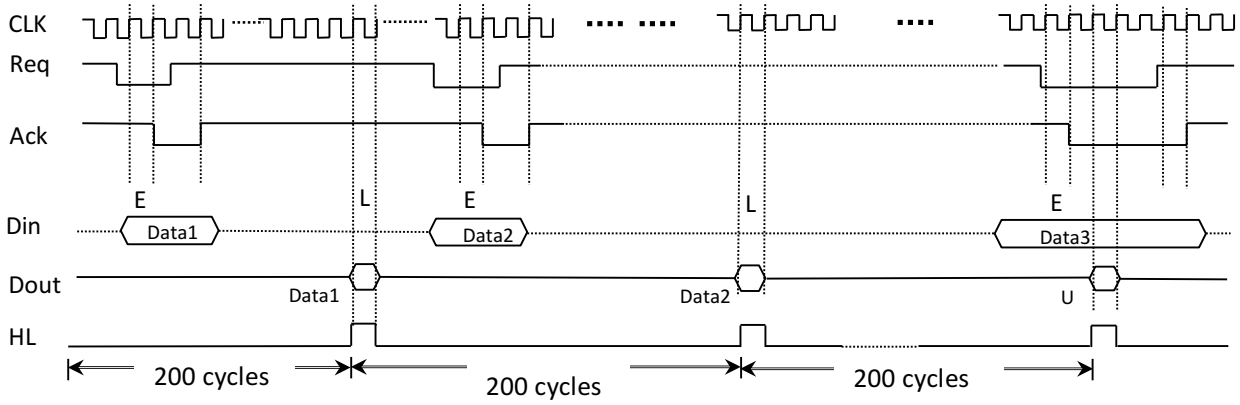


Figure 3

Protocole d'écriture :

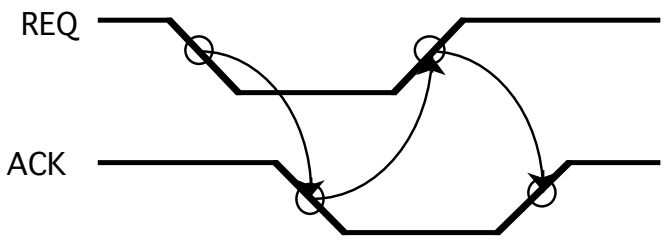


Figure 4

Les besoins :

- Mémoire RAM 2^M x N (M : taille du bus d'adresse, N nombre de bits par mots)
- 4 Compteurs pour :
  - 1 compteur mémorise les adresses des écritures
  - 1 compteur mémorise les adresses des lectures
  - 1 compteur pour diviseur de fréquence (H=10MHz → HL=50 Khz)
  - 1 compteur pour le calcul des signaux "fast" & "slow"
- Multiplexeur M x 2 vers 1
- Séquenceur
- …

La fifo possède 3 paramètres génériques M et N qui déterminent la taille de la RAM. Le nombre de mots est définit par 2^M et N étant le nombre de bits par mots. Le troisième paramètre générique Div est utilisé dans le diviseur de fréquence pour générer l'horloge de lecture HL.

Démarche (A suivre rigoureusement)

Le travail consiste à dans un premier temps d'étudier l'architecture proposée ainsi que le rôle de chaque bloc *(la hiérarchie et l'architecture globale du circuit sont montrées à la fin du document)* puis d'effectuer la description vhdl des différents blocs constituant le composant final qui est la fifo élastique.

Description des blocs de base

En reprenant le projet créé en TD, effectuer la description en VHDL et la validation par simulation des différents blocs un à un dans l'ordre donné ci-dessous. Les détails et le cahier des charges de chacun des blocs sont donnés dans les pages suivantes. **Dès la validation du premier bloc, compléter le fichier package (créé en TD) pour déclarer le bloc**. *Ce fichier package doit être mis à jour après la validation d'un bloc. Commentez au maximum votre code.*

Liste des blocs à décrire :

- La mémoire 2^MxN *(si n'est pas encore fait en TD),*
- Compteur/décompteur de N bits,
- Le bloc GENHL Diviseur de fréquence (10 MHz → 50 KHz)
- Multiplexeur M x 2 vers 1,
- Le bloc Genadr, (Réutilisation du compteur et multiplexeur)
- Le bloc Fastslow (Réutilisation du compteur plus une logique)
- Le séquenceur (à partir d'un graphe fourni)
- Le registre (voir exemple dans le cours)
- Le complément à 2 (code fourni sur Moodle)

Cahier des charges des différents blocs

→ Mettez en entête (commentaires) de tout vos fichiers le nom du fichier et aussi vos noms

1. La mémoire RAM

Faites la description d'une RAM paramétrée N nombre de bits par mot et M la taille du bus d'adresse selon la figure 5

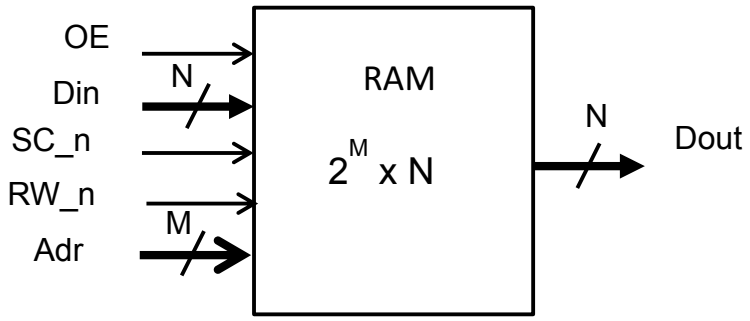


Figure 5

CS_n : Chip Select actif à l'état 0 : Si CS_n = 0 Mémoire valide : on peut lire ou écrire. Si CS_n = 1 mémoire non valide : Dout = haute impédance (en mode lecture et en mode écriture). Ecriture et lecture n'est pas possible

RW_n : Read Write: RW_n=0 => écriture, RW_n=1 lecture

OE : Output Enable : contrôle du bus de données en sortie en mode lecture (à tester seulement en mode lecture) :
si OE=1 Dout = Valide sinon Dout = haute impédance

Générez le fichier testbench puis testez la RAM en écrivant 2 ou 3 données dans des cases mémoires différentes puis les relire. Vérifiez aussi les états de haute impédance du bus de sortie quand CS_n et OE ne sont pas actifs et aussi lors d'un cycle d'écriture.

2. Le Compteur/decompteur

Selon l'architecture globale du circuit on aura besoin de 3 compteurs et 1 compteur-décompteur:

- 2 compteurs M bits pour les pointeurs d'adresses (en mode comptage seulement) (bloc genadr),
- 1 compteur 8 bits pour genHl..(en mode comptage seulement)
- 1 compteur-décompteur M bits pour calculer fast & slow (mode comptage et décomptage),

On fera la description d'un seul compteur-décompteur paramétrable par un paramètre générique avec une entrée de validation et on utilisera la même instance compteur-décompteur lors de la description des niveaux hiérarchiques supérieurs.

Effectuez la description et le test d'un compteur décompteur de M bits avec un ***reset synchrone*** et une entrée de validation selon la figure 6.

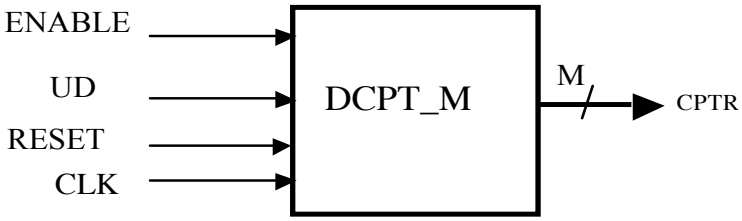


Figure 6

3. *Le générateur horloge de lecture Hl (GENHL) :*



Figure 7

Ce module permet de calculer les deux signaux complémentaires ENREAD (*Enable Read*) et ENWRITE (*Enable Write*). Ces deux signaux indiquent au séquenceur à quel moment on **peut** faire une écriture (asynchrone) et à quel moment on **doit** faire une lecture (synchrone).

Pour décrire ce bloc, il faudra instancier le compteur décompteur décrit précédemment avec la valeur du paramètre générique à 8 puis lui associez une logique combinatoire. La logique combinatoire doit non seulement générer les 2 signaux ENREAD et ENWRITE mais aussi remettre à 0 le compteur une fois qu'il aura fait 200 cycles. Il ne faut pas oublier de prendre en compte le signal reset du système.
Remarque : il n'est pas nécessaire de faire un composant pour faire la partie combinatoire. On peut rajouter des instructions concurrentes avec l'instruction d'instanciation.

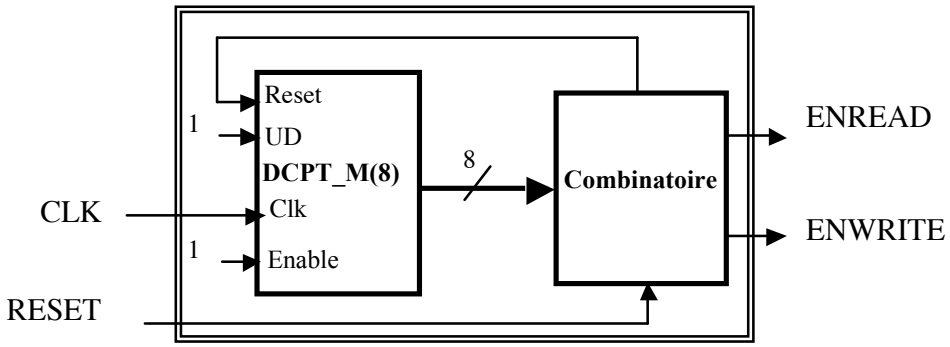


Figure 8_a

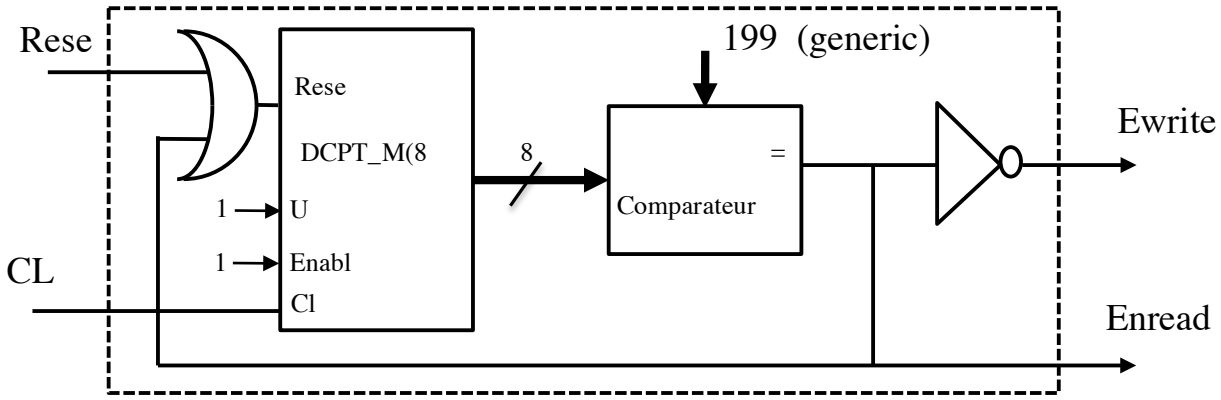


figure 8_b

4. *Le Générateur d'adresse*

Le générateur d'adresse utilise 2 compteurs M bits et un mux M x 2 → 1 comme montré en figure 9b. Utilisez le compteur-décompteur décrit précédemment et le multiplexeur Mux 2 → 1. Il n'est pas nécessaire de faire un composant mux2→1.

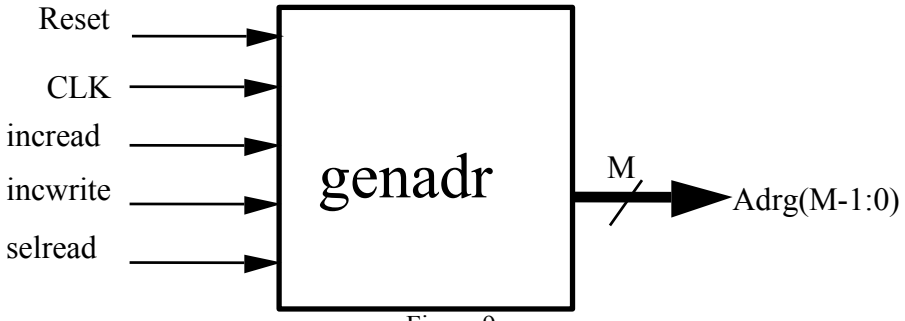


Figure 9a

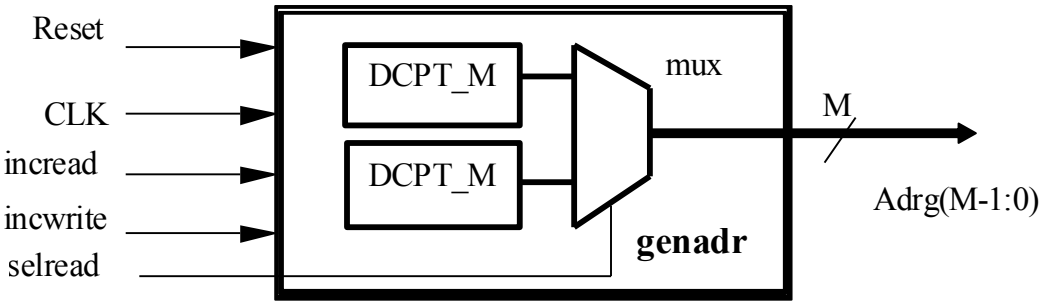


Figure 9b

5. Fastslow

Ce module utilise un compteur-decompteur M bits. Il est incrémenté au front montant de l'horloge si *incwrite* est à 1 et il est décrémenté au front montant de l'horloge si *incread* est 1. Une petite logique combinatoire à la sortie du compteur/décompteur permet de calculer les deux signaux *fast* et *slow*.

Afin d'éviter que la FIFO se vide ou déborde ou elle se vide, les deux signaux *fast* et *slow* sont générés. Dans d'une fifo de 16x8, le signal *fast* est à 1 s'il y a **moins de 4 données**, de même un signal *slow* est généré si la fifo contient **12 données ou plus**. Ces valeurs sont valables uniquement pour une fifo 16x8 (16 mots de 8 bits). Dans le cas général, on prendra les conditions suivantes :

Fast vaut 1 si le nombre de mots dans la fifo est $< 2^{M-2}$
Slow vaut 1 si le nombre de mots dans la fifo est $\geq 2^{M}-2^{M-2}$
2^{M} étant le nombre de mots max.

Ne vous lancez pas à implémenter ces deux expressions impliquant des comparaisons exponentiation et soustractions. Il y a bien des astuces qui vous permettent de calculer les deux signaux fast et slow juste avec une seule porte logique chacun.

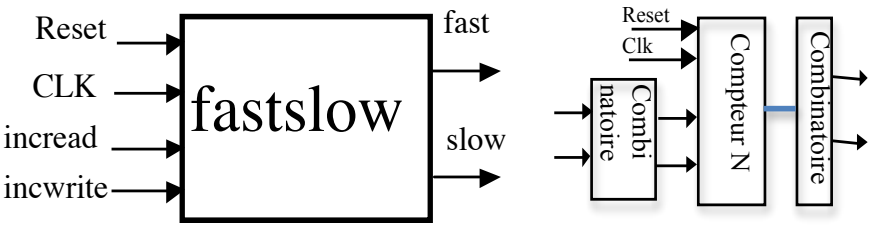


Figure 10

6. *Les blocs « Complément à 2 » et le registres N bits.*

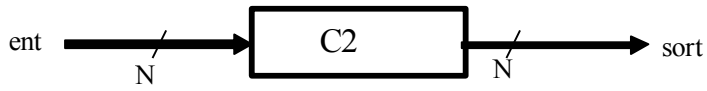
Ces deux blocs nous permettront des faire des appels de Fonctions (calcule complément à 2) et des appels de procédures (vérification des temps de pré positionnement et de maintien d'un registre N bits).

➢ *Le bloc complément à 2*

Pour se familiariser avec le mécanisme des sous-programmes en VHDL, dans un premier temps, on commencera par la description d'un opérateur simple en faisant appel à une fonction. La seconde partie consiste en la mise au point des procédures pour la vérification du temps de pré-positionnement et le temps de maintien (voir annexe) d'un registre N bits.

- On veut décrire un composant effectuant le complément à 2 d'un signal mais le calcul doit se faire par l'appel d'une **fonction**.

a) La fonction nommée ″**cpl2**″effectuant le complément 2 est fournie en annexe et en ligne. Les paramètres passés à la fonction sont le signal dont on veut calculer le complément à 2 et la taille de ce signal. Rajoutez le fichier complement_a_2.vhd à vottre projet et le package mes_fonctions (fichier mon_package) au le projet puis compilez.



b) Simuler le composant C2.

➢ *Le Registre N bits*

Le registre que l'on veut décrire ici n'est pas un registre simple. Il s'agit d'un registre capable de mesurer et vérifier si les **temps de pré positionnement** (**set up time**) et de maintient **(hold time)** sont respectés ou non.
On a besoin de faire ces vérifications lorsqu'on modélise des éléments de mémorisation.
La mesure et la vérification doivent se faire à l'aide des appels à des procédures. Vous trouverez les codes de ces procédures en annexe du sujet de ce TP et ils sont aussi sur Moodle. La **procédure check_hold** est à compléter (voir poly).

a) Dans l'architecture du registre, appelez la **procédure check_setup** pour vérifier le temps de pré positionnement. Dans le scénario du test du registre, provoquez des cas de non respect du temps de pré positionnement. Si le temps de pré positionnement n'est pas respecté, un message "T_setup non respecté" doit être affiché à l'écran lors de la simulation.

b) Dans l'architecture du registre, appelez la **procédure check_hold** pour vérifier le temps de maintien. Dans le scénario du test du registre, provoquez des cas de non respect du temps de maintien. Si le temps de pré positionnement n'est pas respecté, un message "T_hold non respecté" doit être affiché à l'écran lors de la simulation.

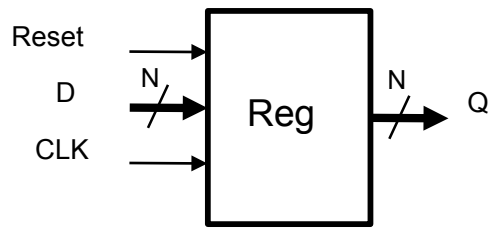


Figure 12

7. *Le séquenceur*

Le séquenceur échantillonne les entrées au front montant de l'horloge. Il s'occupe de calculer les signaux de commandes pour le reste du circuit. Commencez par faire le graphe, puis la description.

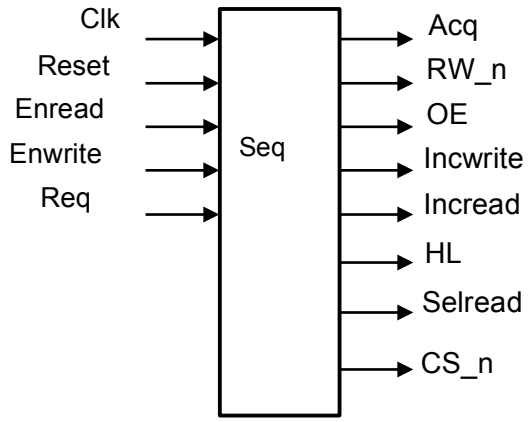


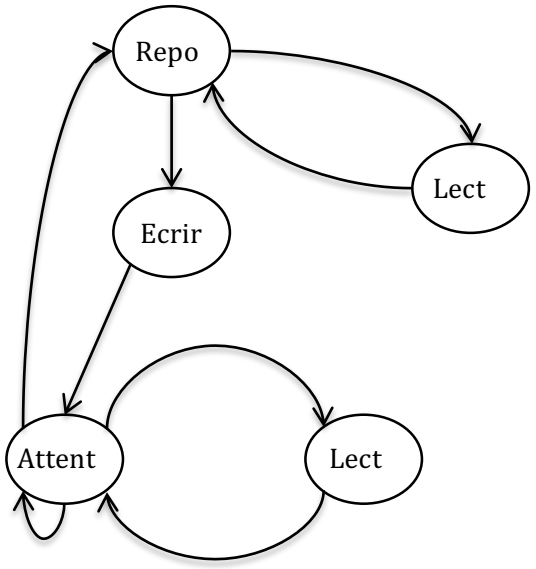
Figure 13

Dans un même fichier et pour une **même entité du séquenceur** faites la description de 2 architectures :

- archi_Mealy
- archi_Moore

Graphe à Compléter

- (conditions de transition)
- Les sorties en fonction des états
  Il est conseillé de faire un tableau (Etats, Sorties)



Simulation :

Simulez le séquenceur de façon à vérifier les transitions entre les états et les valeurs des sorties et aussi le passage par l'ensemble des états en fonction des entrées : *clk, reset, req, enread et enwrite.*

Remarque : Il est très intéressant de visualiser les états avec les signaux en les sélectionnant en mode simulation dans la fenêtre « Objects » si le séquenceur est sélectionné dans la fenêtre « Instance ».

	Ack	**RW_n**	**OE**	**Incwrite**	**Incread**	**HL**	**Selread**	**CS_n**
Repos	1					0		
Ecrire	0					0		
Lect1	1					1		
Attente	0					0		
Lect2	0					0		

8. Interconnexion des blocs et simulation de la FIFO

Rassemblez tous les composants pour implémenter la FIFO complète dans un fichier FIFO.vhd. Vérifier que tous les blocs sont déclarés dans le package. Recenser le nombre des signaux internes nécessaires à déclarer et identifier les paramètres génériques de chaque bloc. Commentez au maximum votre code. Utiliser les valeurs par défaut des paramètres générique de façon à avoir une FIFO de 16 mots de 8 bits et un rythme de lecture de 250 cycles.

Une fois le compilée avec succès, générez le fichier testbench et concevez un scénario de façon à faire au moins trois requêtes d'écriture dans la fifo puis laissez tourner la simulation au moins 800 cycles afin d'observer 3 lectures. Vérifier les 3 données lues sur bus Dout par rapport au contenu de la RAM et les données mises sur l'entrée Din.

9. Simulation avec un fichier de configuration

Sachant que le séquenceur possède 2 architectures, éditez un fichier de configuration pour simuler la FIFO et de spécifier lequel des 2 architectures (Mealy ou Moore) utiliser.

10. Synthèse de la FIFO

La synthèse peut être effectuée soit avec Quartus soit avec Vivado. Créez un nouveau projet sur dans Quartus ou Vivado. Choisissez comme FPGA cyclone V 5CSEMA5F31C6 pour la carte DE1-SOC ou bien Artix-7 XC7A35T-ICPG236C pour Vivado. Ajoutez vos fichiers VHDL (ne pas rajouter les fichiers testbench) puis effectuez la synthèse de la FIFO (16x8). Notez la complexité et la fréquence maximale de fonctionnement.

11. Teste de la FIFO sur la carte FPGA

Si vous êtes bien avancé, vous pouvez concevoir et décrire un composant qui permet de fournir les données et implémenter le protocole req-ack. (un composant qui remplace le fichier testbench)

En diminuant (division) la fréquence de fonctionnement et en connectant le bus Dout sur des LED de la carte FPGA on pourra observer le changement des données sur les LEDs. On peut aussi utiliser une horloge externe à la carte (si ma carte le permet) avec une fréquence faible pour pouvoir observer le changement des données sur les LEDs.

Pour cela, créez un nouveau composant dans lequel vous instanciez la FIFO et votre composant ou bien le composant Fifo_data_supply (fichiers sources fournies sur Moodle). Le rôle du composant Fifo_data_supply une fois connecté à la FIFO permet de lui fournir les données et implémenter le protocole req-ack. Après avoir analysé les fichiers du composant Fifo_data_supply, effectuez une simulation puis une implémentation sur la carte FPGA DE1-SOC ou Basys3.

12. Compte rendu

A la fin de la dernière séance, il faut déposer sur Moodle (date et horaire limite à préciser) le projet complet compressé (Fichiers sources VHDL y compris les fichiers de test et celui de la configuration (VHDL)) dont le nom doit respecter le format suivant : *TP1_VHDL_IS_Nom1_Nom2.zip*

Le compte rendu (à déposer un peu plus tard) doit contenir :

- Les chronogrammes de simulation des différents blocs et celui de la FIFO Complète avec les deux architectures du séquenceur Mealy et Moore avec explication et commentaires.

- Le rapport de synthèse de la FIFO seule et celui de la FIFO avec Data_supply

13. Annexes (ces fichiers sont fournis sur Moodle)

```
-------------------------------------------------------------------
    -- complement_a_2.vhd
-------------------------------------------------------------------
library ieee ;
use ieee.std_logic_1164.all ;
use work.mes_fonctions.all ;

entity complement_a_2 is
generic (N : natural  := 8) ;
port (nombre : in std_logic_vector( N-1 downto 0) ;
       sortie : out std_logic_vector( N-1 downto 0)) ;
end complement_a_2 ;

architecture archi_complement_a_2 of complement_a_2 is
begin
sortie <= cpl2(nombre, N) ;
end archi_complement_a_2 ;
```

```
-------------------------------------------------------------------
    -- mon_paquetage.vhd
-------------------------------------------------------------------
library ieee ;
use ieee.std_logic_1164.all ;
use ieee.std_logic_unsigned.all ;

package mes_fonctions is
function cpl2 (entree : std_logic_vector ; N : natural)
                                    return std_logic_vector ;
end mes_fonctions ;
-------------------------------------------------------------------
library ieee ;
use ieee.std_logic_1164.all ;

package body mes_fonctions is
function cpl2 (entree : std_logic_vector ; N : natural)
                                    return std_logic_vector  is

variable temp : std_logic_vector( N-1 downto 0) ;
begin
temp := not entree;
temp := temp + '1' ;
return temp ;
end cpl2 ;
end mes_fonctions ;
-------------------------------------------------------------------
library ieee ;
use ieee.std_logic_1164.all ;

package CHECK_PKG is

-- Déclaration de la procédures check_setup.
procedure check_setup( signal clk    : in std_logic;
                               signal din    : in std_logic_vector;
                                     t_setup        : in time;
                                     severite       : in severity_level:= warning;
                                     hdeb           : in time := time'low;
                                     hfin           : in time := time'high );


procedure check_hold          ( signal clk   : in std_logic;
                               signal din    : in std_logic_vector;
                                     t_hold  : in time;
                                     severite       : in severity_level:= warning;
                                     hdeb           : in time := time'low;
                                     hfin           : in time := time'high );
end CHECK_PKG;
-------------------------------------------------------------------
library ieee ;
use ieee.std_logic_1164.all ;

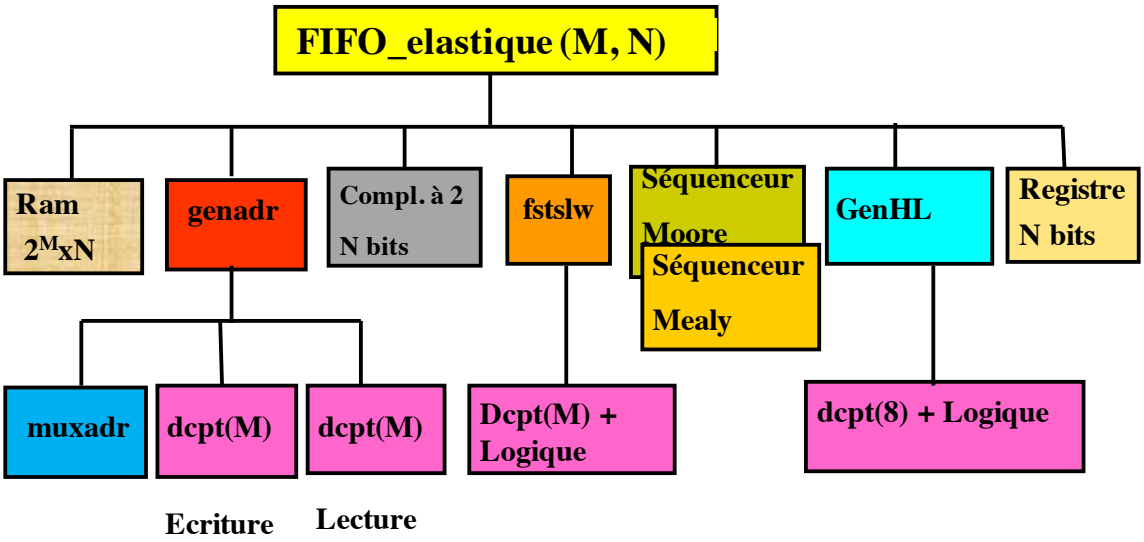
package body CHECK_PKG is
-- Spécification du corps du paquetage.
--la procédure check_setup
procedure check_setup( signal clk    : in std_logic;
                              signal din     : in std_logic_vector;
                                     t_setup        : in time;
                                     severite       : in severity_level:= warning;
                                     hdeb           : in time := time'low;
```

```
                                    hfin          : in time := time'high) is
begin
loop
wait on clk;
If clk = '1' and clk'event then
        if now >= hdeb and now <= hfin then
        assert din'last_event >= t_setup and din'event = false
        report "temps de setup non respecté" severity severite;
        elsif now > hfin then
        wait;
        end if;
end if;
end loop;
end check_setup;
procedure check_hold  ( signal clk   : in std_logic;
                        signal din    : in std_logic_vector;
                               t_hold         : in time;
                               severite       : in severity_level:= warning;
                               hdeb           : in time := time'low;
                               hfin           : in time := time'high ) is
        variable t : time;

begin
loop
wait until clk='1';
t:= now;
        if t>= hdeb and t<= hfin then
                if din'event = false then
                        wait on din for t_hold;
                end if;
        assert din'event = false or (now - t) = t_hold
        report "temps de hold non respecte" severity severite;
        elsif t > hfin then
        wait;                   -- suspension definitive.

        End if;
End loop;
End check_hold;
End CHECK_PKG;
```

14. Hierarchie de la fifo elastique



15. Architecture de la fifo élastique

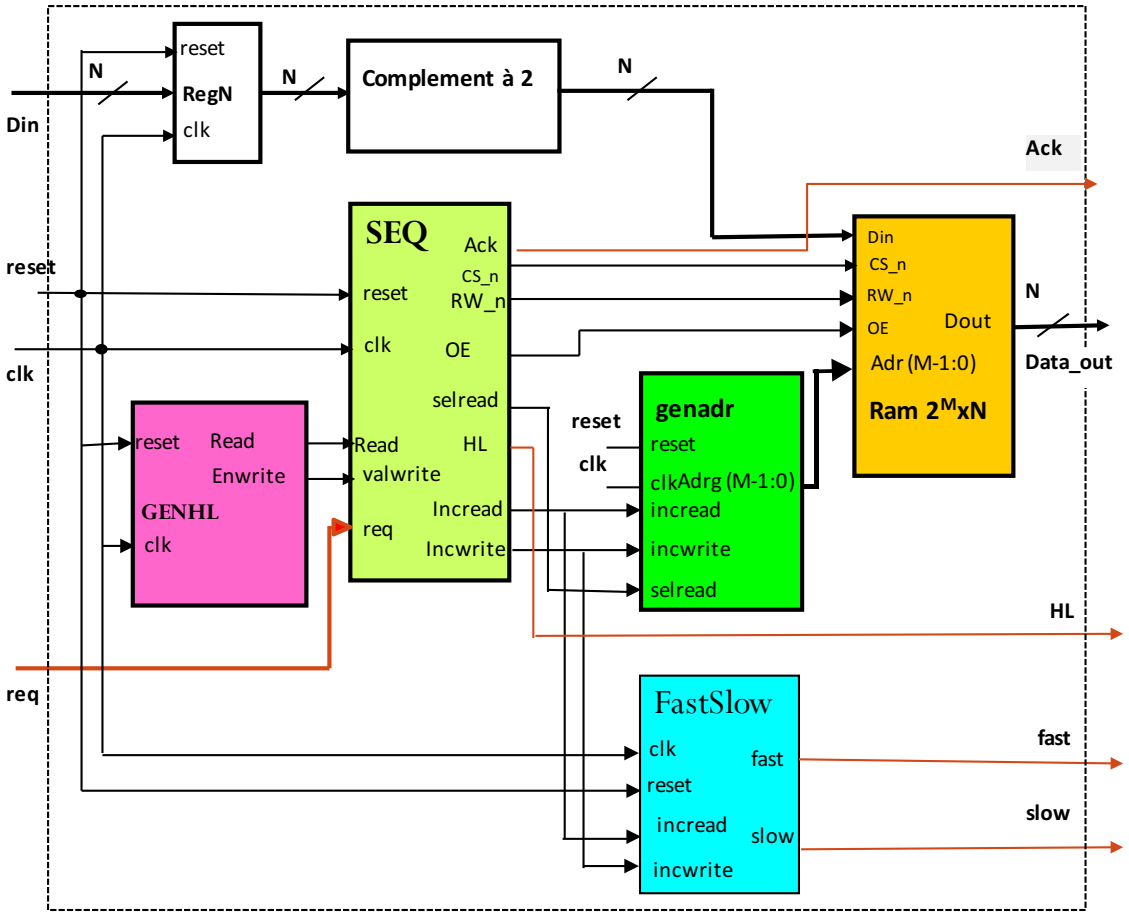


Table des matières

Objectifs : 1

Cahier des charges 1

Les besoins : 3

Démarche (A suivre rigoureusement) 3

Description des blocs de base 3

Liste des blocs à décrire : 3

Cahier des charges des différents blocs 3

1. **La mémoire RAM** 3
2. **Le Compteur/decompteur** 4
3. **Le générateur horloge de lecture Hl (GENHL) :** 5
4. **Le Générateur d'adresse** 5
5. **Fastslow** 6
6. **Les blocs « Complément à 2 » et le registres N bits.** 7
   - **Le bloc complément à 2** 7
   - **Le Registre N bits** 7
7. **Le séquenceur** 8
8. **Interconnexion des blocs et simulation de la FIFO** 8
9. **Simulation avec un fichier de configuration** 9
10. **Synthèse de la FIFO** 9
11. **Teste de la FIFO sur la carte FPGA** 9
12. **Compte rendu** 9
13. **Annexes (ces fichiers sont fournis sur Moodle)** 10
14. **Hierarchie de la fifo elastique** 12
15. **Architecture de la fifo élastique** 12

TP1 : Conception circuits numériques : Méthodologie de Conception & Langages de description de matériel (VHDL) Compléments

FIFO Elastique

4 séances de 4h

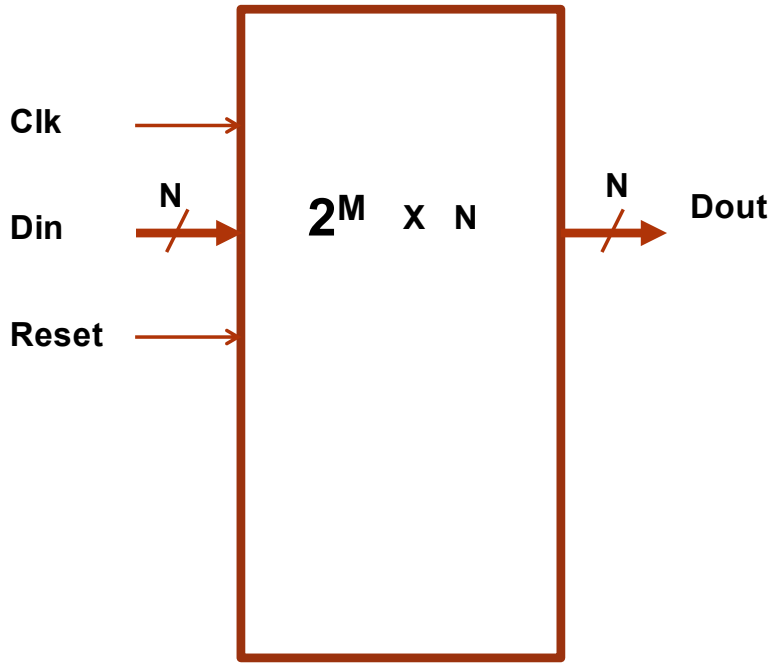
2025 -2026

Objectifs

- Méthodologie & Aspects avancés de VHDL
  - Modélisation par paramètres génériques des composants de base
  - Packages
  - Séquenceurs
  - Configuration
  - Sous-Programmes (fonctions et procédures)
  - Assertions (affichage des messages lors de la simulation)
  - Attributs (signaux)
  - Synthèse et implémentation sur carte FPGAar

FIFO SIMPLE

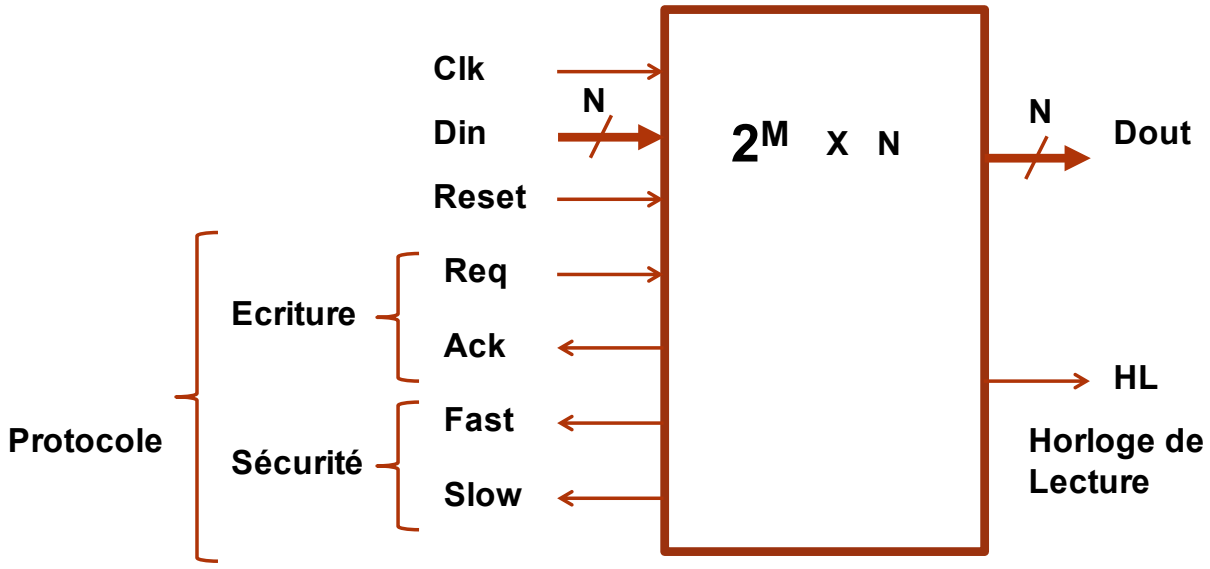
2^M Mots de N bits



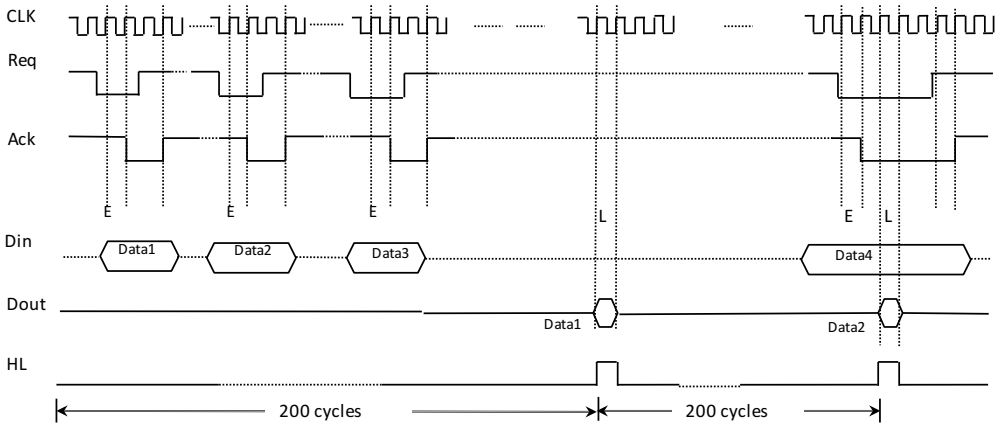
FIFO ELASTIQUE ?

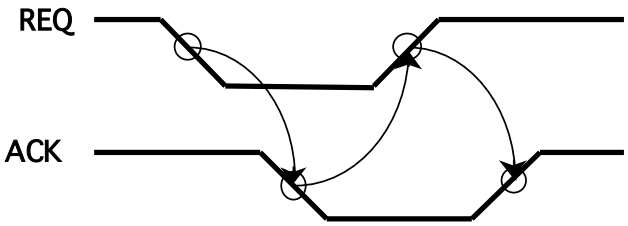
- FIFO : First In First Out
- ELASTIQUE :
  - ECRITURE **ASYNCHRONE**
    - → **protocole**
  - **LECTURE SYNCHRONE**
    - → **synhrone avec une horloge**

FIFO ELASTIQUE 2^M mots de N bits



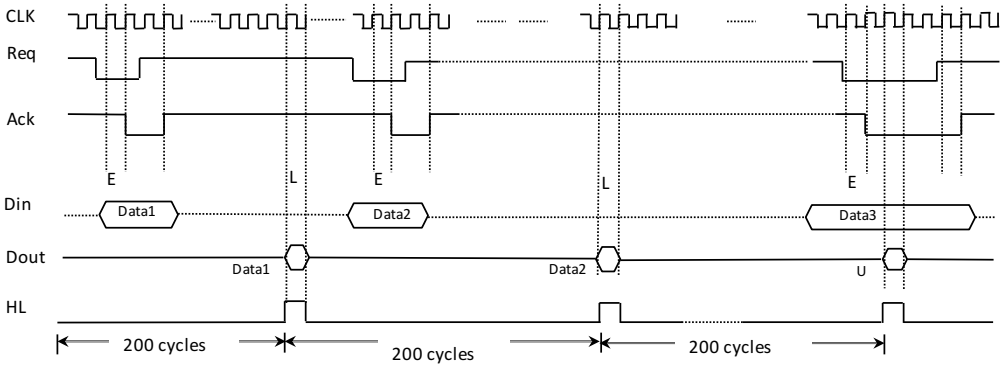
Cahier des charges en courbes (suite)

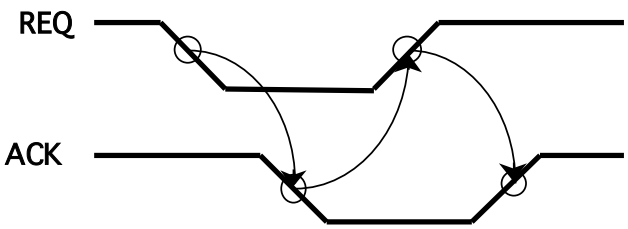




Cahier des charges en courbes

Rappel :



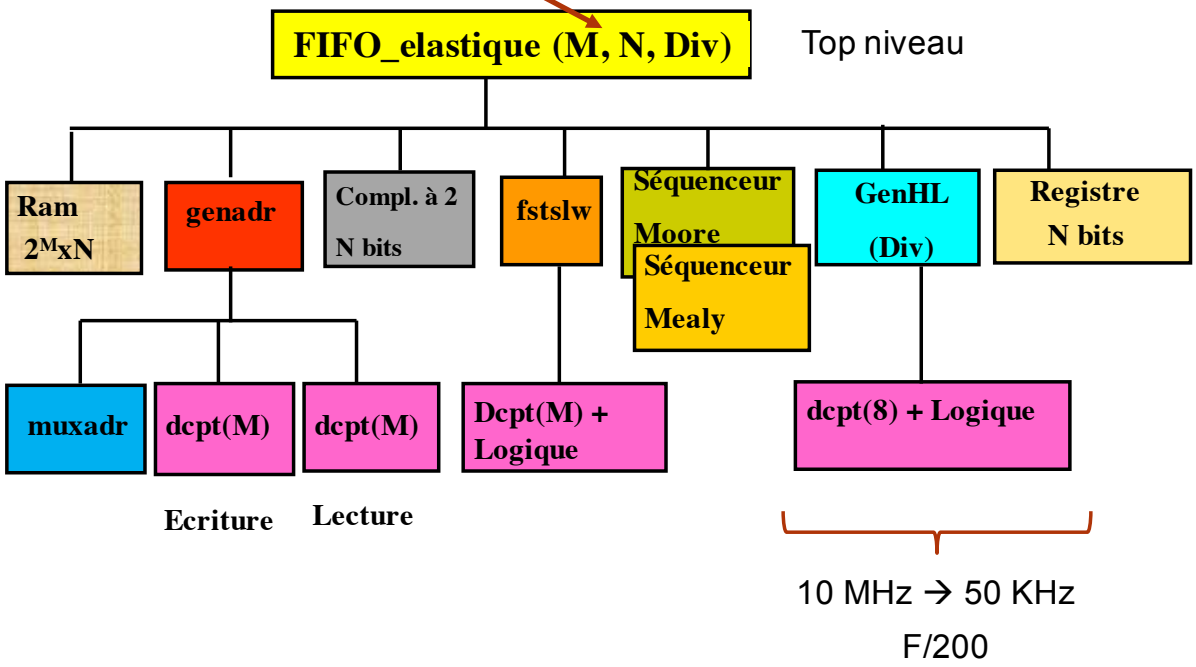


Besoins

- RAM : 2^M x N bits
- Générateur d'adresse
  - 2 compteurs
  - 1 multiplexeur
- Compteur-décompteur (Fast – Slow)
- Signal de lecture périodique (GenCLKRead 10 MHz → 50 KHz)
  - Compteur + Logique
- Séquenceur
- +…

Hiérarchie

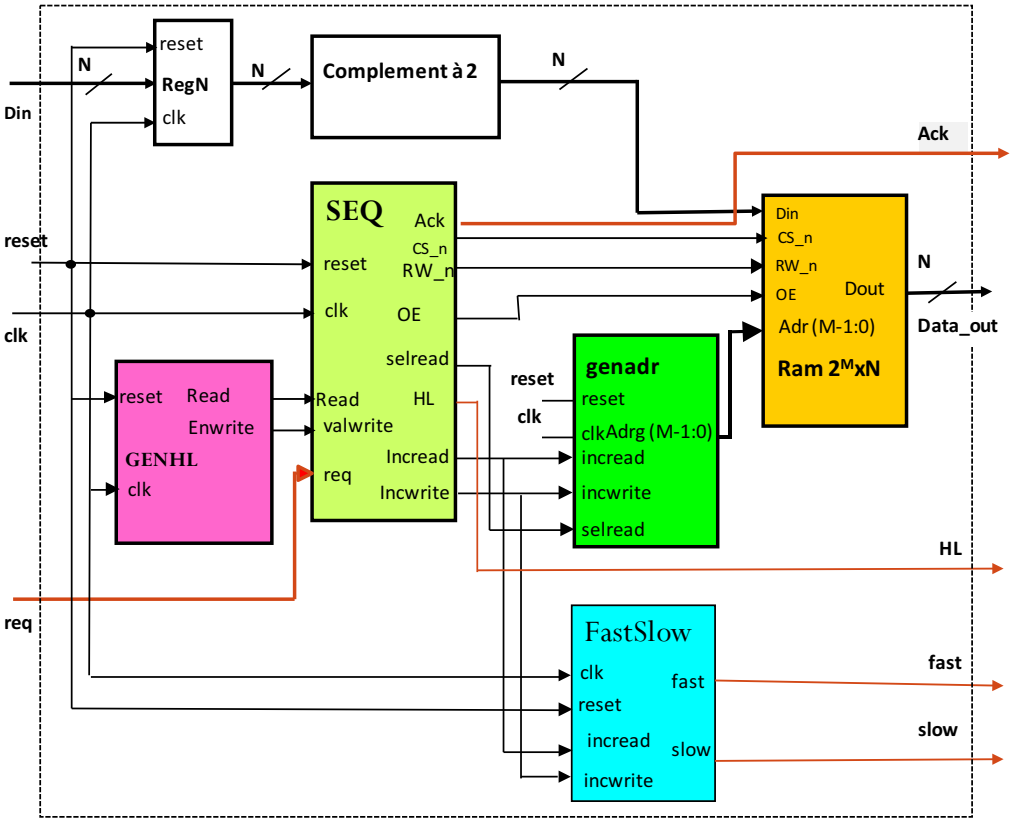
2 Paramètres génériques



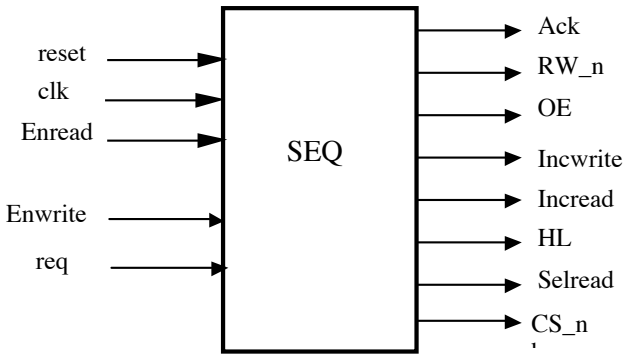
10 MHz → 50 KHz

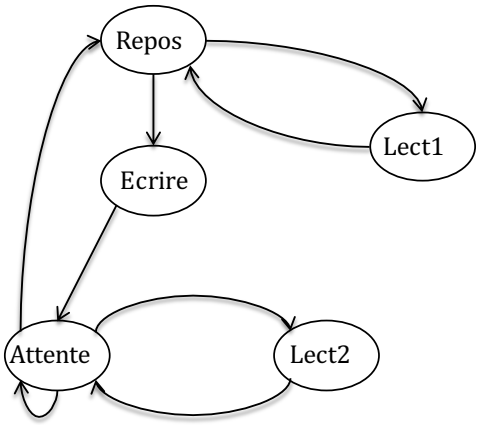
F/200

Architecture (niveau 2)



Le séquenceur





	Ack	RW_n	OE	Incwrite	Incread	HL	Selread	CS_n
Repos	1					0		
Ecrire	0					0		
Lect1	1					1		
Attente	0					0		
Lect2	0					0		

11

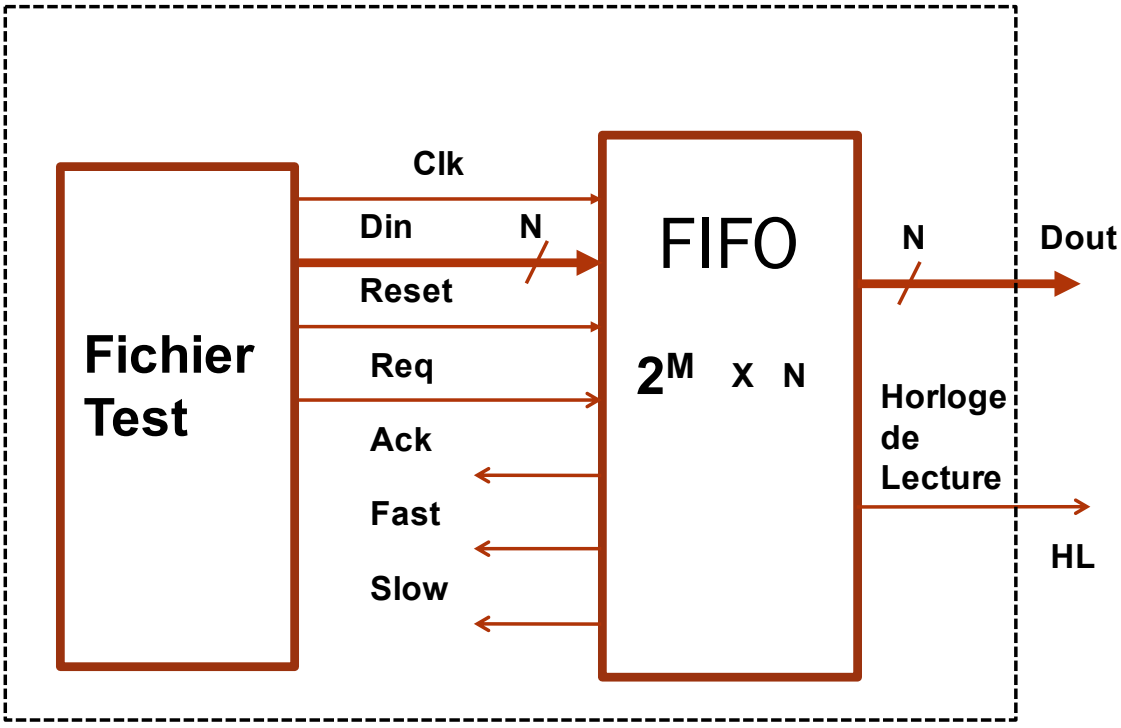
Démarches

- Création d'un projet avec Modelsim
- Création d'un fichier package :
  - Exemple : mes_composants.vhd, mon_package.vhd, …
- Pour chaque composant :
- Description du Composant
  - Compilation
  - Déclaration dans le package
- Création du Fichier de Test associé puis complété par le scénario
  - Compilation
  - Simulation
  - ….
- Composant Top (Fifo elastique) (assemblage des blocs)
- Compilation
- Création du Fichier de test associé + Scénario + simulation
- Configuration
- Simulation avec la configuration

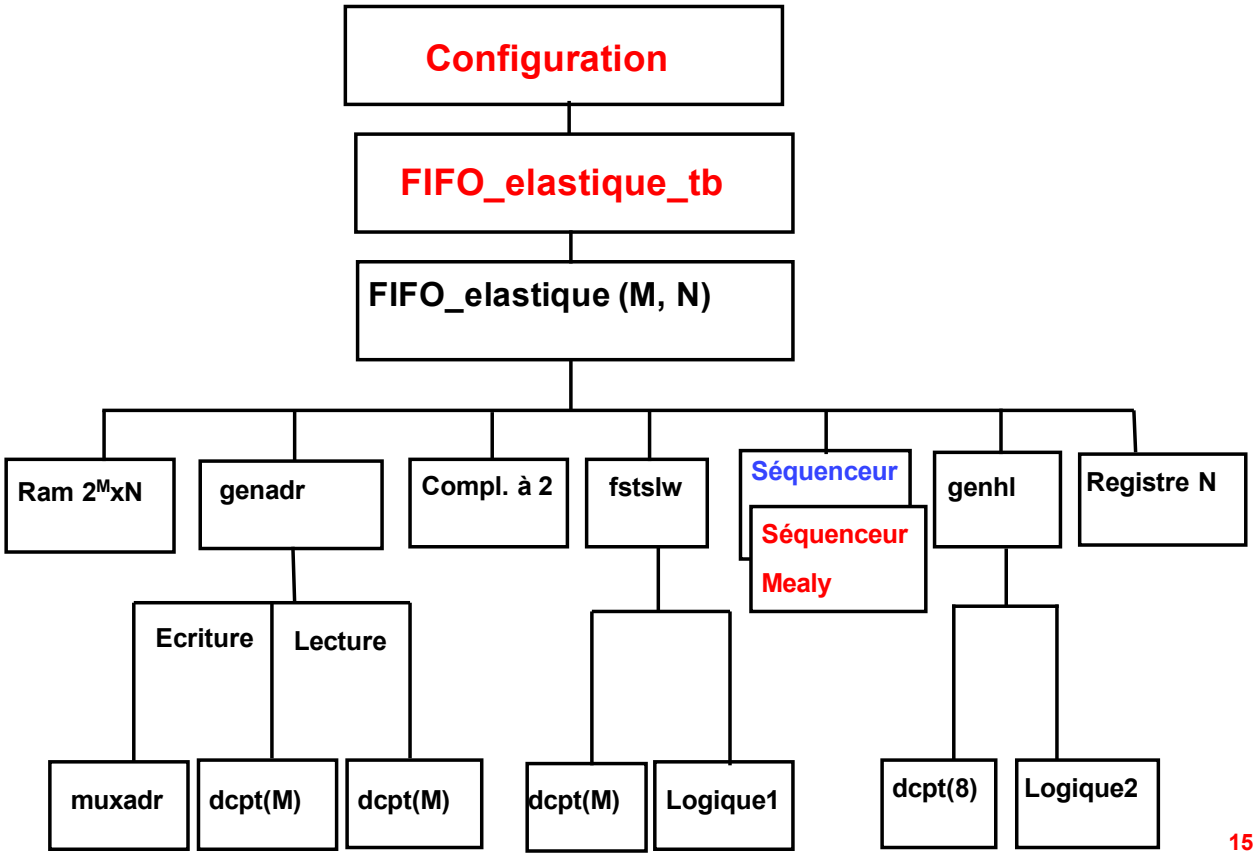
Démarche suite

- Création d'un projet avec Quartus ou Vivado
- Synthèse de la fifo toute seule
- Conception circuit fournisseur de données (Data_supply)
- Simulation du composant Data_supply
- Faire un composant : Fifo + Data_supply
- Simulation Fifo + Data_supply
- Implémentation sur carte FPGA

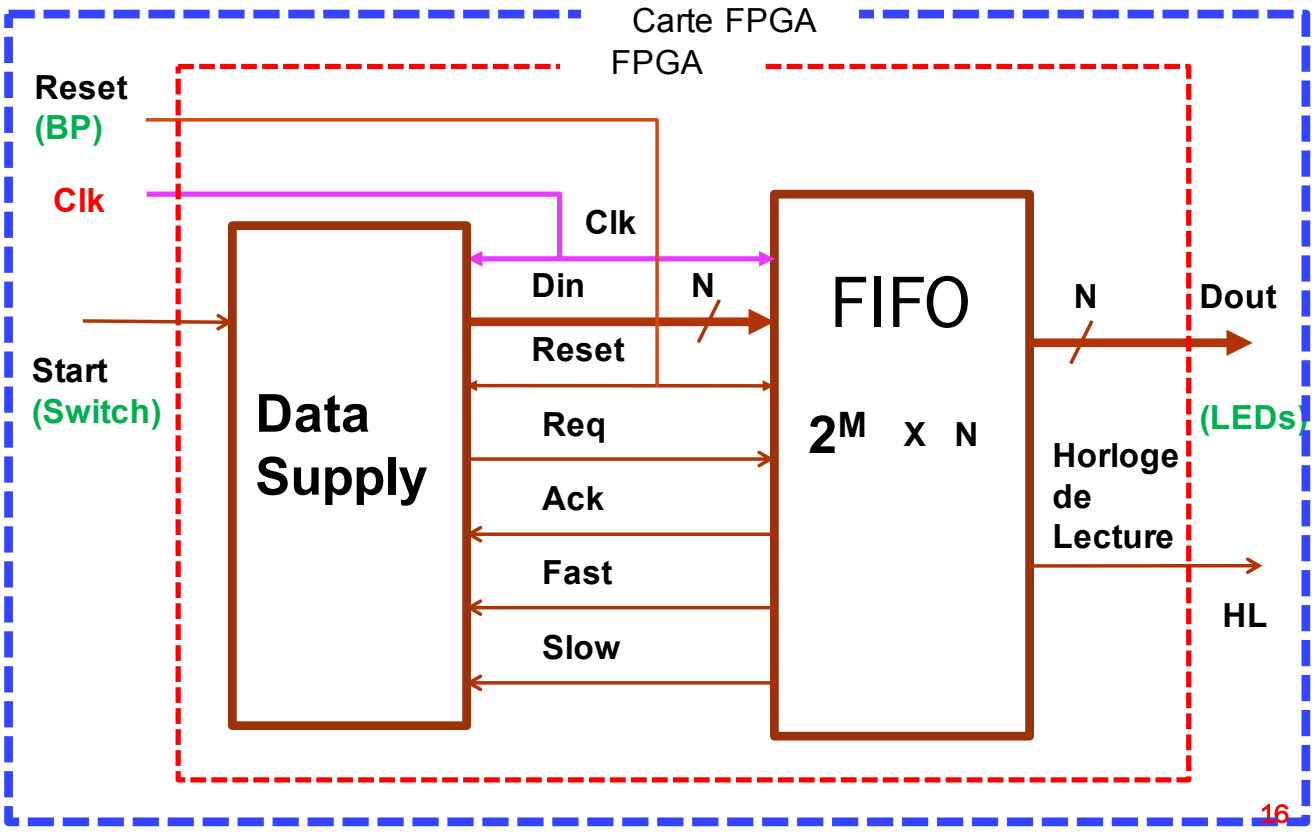
FIFO ELASTIQUE 2^M mots de N bits



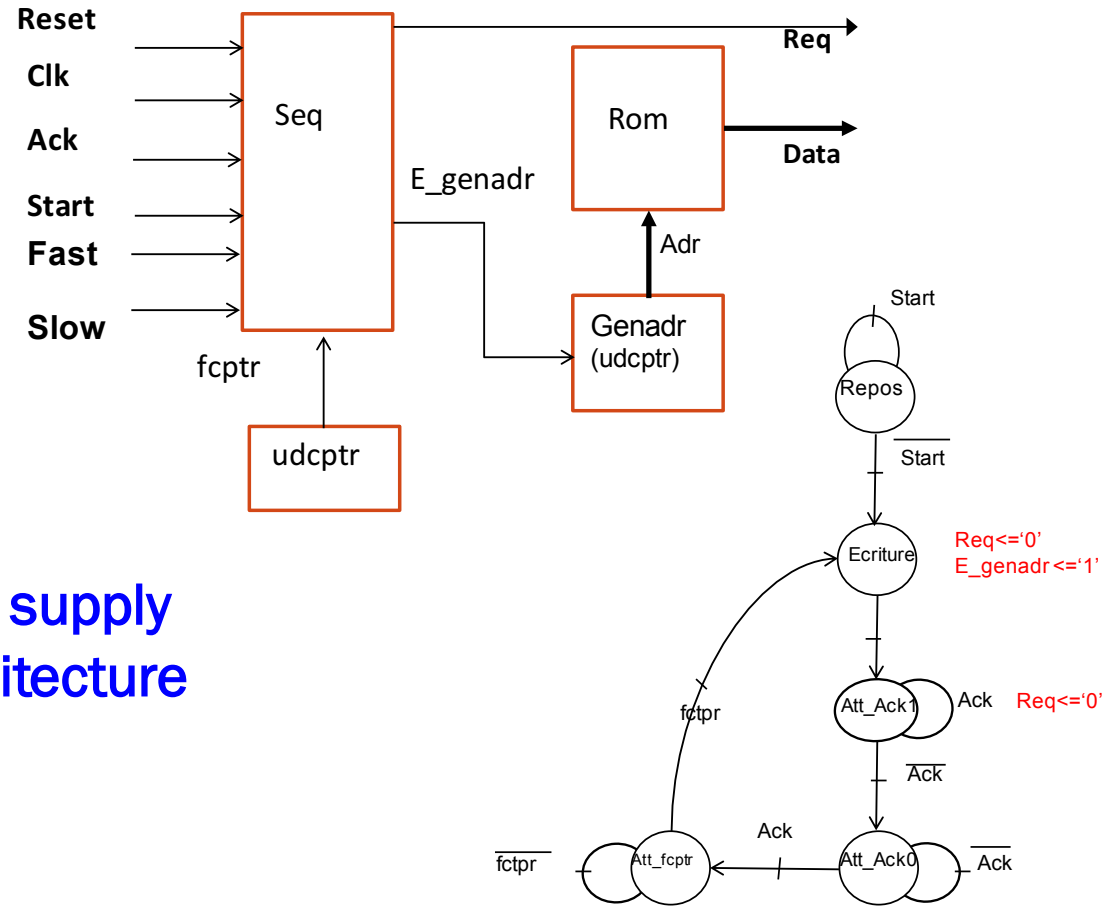
Hiérarchie



FIFO ELASTIQUE 2^M mots de N bits



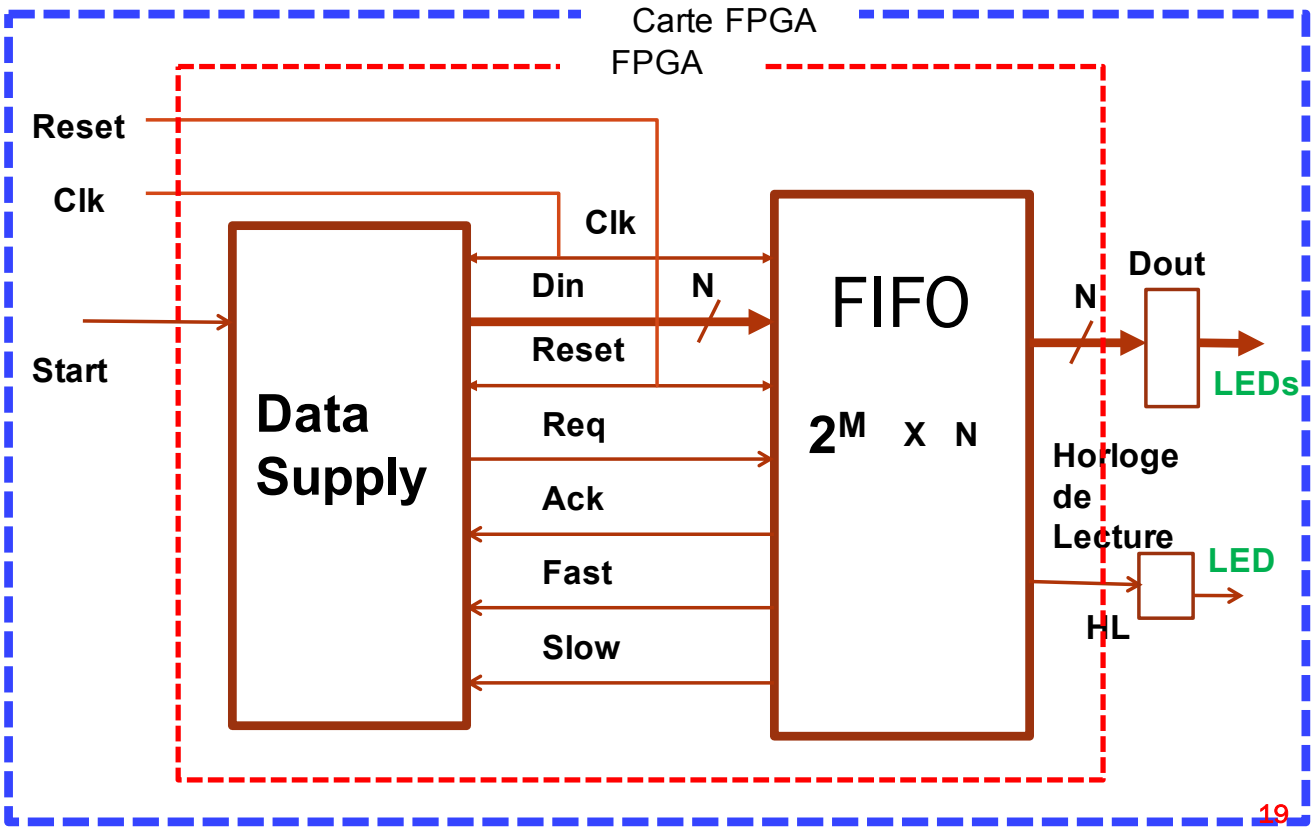
Data supply Architecture



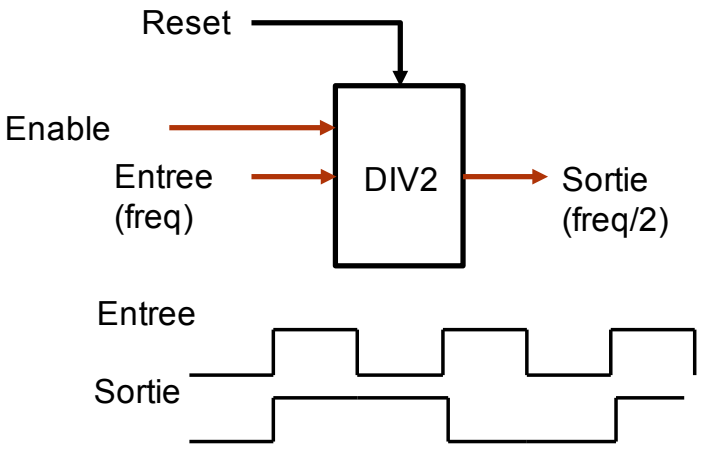
Horloge ?

- Horloge sur la carte → 50 MHz ou 100 MHz
- → Injecter une horloge Basse Fréquence si la carte le permet
- → Sinon utiliser un diviseur de fréquence
  - DIV2
  - Cascader N fois DIV2 → $DIV2^n$
- Rajouter un registre après Dout pour maintenir les données entre 2 lectures

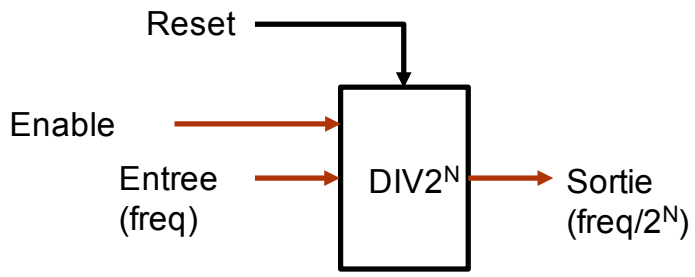
FIFO ELASTIQUE 2^M mots de N bits



Faire un composant → DIV2



DIV2N



50 MHz → ≈10 Hz

Les outils et carte utilisés

Modelsim :
→ Mentor graphics : Licence étudiant gratuite (limitée)
ou → **ModelSim-Altera 10.1d (Quartus II 13.0 sp1)**

Quartus prime 18.1 → Altera → Carte DE1-SOC (Cyclone V)

Ou Vivado 2019 → Basys3 ou autre